ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #21

TaskCraft v0.1.2

Local-first issue tracker for a small team, with git commit linking. One SQLite file, one binary that is both CLI and MCP server. Part of the back-office craft suite (replaces Linear / Jira / Asana for the 1-10 person case).



v0.1.2	3 Crates	3191	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\taskcraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

Local-first issue tracker for a small team, with git commit linking. One SQLite file, one binary that is both CLI and MCP server. Part of the back-office craft suite (replaces Linear / Jira / Asana for the 1-10 person case).

Status: **v0.1.2, Tier 1** (store + CLI + MCP, tested). No desktop window yet. Synthetic demo data only; not used on a real team yet.

- How to use it day to day: HELP.md
- What it is for and where it stops: docs/PRD.md
- Every MCP tool: docs/TOOLS.md (generated, checked by a test)
- Changes: CHANGELOG.md

What it does

- Projects with a short key; issues numbered `KEY-n` per project.
- Status lanes: backlog, todo, in_progress, in_review, done, canceled. Priority low..urgent, story points, assignee, labels, due date, milestones with progress.
- Relations: `blocks` (cycle-checked), `relates_to`, `duplicates`. Starting or finishing a blocked issue is refused unless forced.
- Full per-issue history (who changed what, including agents, git and other craft apps).
- `git-sync`: reads the project repo's `git log`, links commits that mention `KEY-n`, closes issues on `closes/fixes/resolves KEY-n`. Idempotent.
- Inbox: `ticket.escalated` from deskcraft and `contract.deadline` from contractcraft become issues automatically.
- Inbox: `capacity.changed` from teamcraft is stored per ISO week and shown in `status` (available team hours).

Quick start

```
taskcraft demo-seed                 # labelled demo project BC (empty DB only)
taskcraft board BC
taskcraft new BC "Fix CSV export" --priority high --labels bug,export --estimate 2
taskcraft link BC-4 blocks BC-5
taskcraft move BC-5 in_progress     # refused: blocked by BC-4, BC-6
taskcraft project create TC "TaskCraft" --repo D:\Dev\repos\taskcraft
taskcraft git-sync TC --allow-path D:\Dev\repos\taskcraft
taskcraft --json show BC-5
```

Database: `--db PATH`, else `$TASKCRAFT_DB`, else `%LOCALAPPDATA%\taskcraft\taskcraft.db`.

MCP

```
{ "mcpServers": { "taskcraft": { "command": "D:/Dev/cargo-
target/release/taskcraft.exe", "args": ["mcp", "--allow-path", "D:/Dev/repos"] } } }
```

17 tools, all `task_*` (dotted aliases like `task.board` accepted); see docs/TOOLS.md. Agent changes are recorded with actor `agent`.

Build

```
just ci             # clippy -D warnings, tests, fmt check
just build-release
```

Needs `../craft-biz` checked out next to this repo. `git` on PATH for git sync (tests skip the git case if missing).

Not done (honest list)

- No egui desktop app, no drag-and-drop board.
- No comments/attachments on issues; description only.
- No sprints/cycles, no time tracking (teamcraft owns capacity).
- Git sync is pull-on-demand; no hooks, no watcher. GitHub/GitLab issue import not implemented.

Product Requirements & Architecture Specification

What it is

A local-first issue tracker for a 1-10 person team (Linear / Jira / Asana replacement for the small case), with git commit linking and inputs from the other back-office craft apps. Pure Rust, ArtCraft pattern. Suite plan: `BACKOFFICE_CRAFT_SUITE.md`, section 4.2.

Users

- Sandra and a small team planning work per repo or project.
- Agents over MCP: create and triage issues, move them, link commits. Every agent change is in the history with actor `agent`.

Jobs

1. Keep one honest list of work per project, with priorities, estimates and due dates.
2. Make blockers impossible to ignore.
3. Connect work to commits without a hosted service.
4. Collect work that other apps discover (support escalations, contract deadlines).

Today (v0.1.1)

- Projects with keys, `KEY-n` issues, statuses, priorities, points, labels, assignee, due date, milestones with progress.
- Relations with a cycle check on `blocks`; moving a blocked issue to in_progress / in_review / done needs `force`.
- Full per-issue history (agent, user, `git:<author>`, `bus:<app>`).
- `git-sync`: read-only `git log`, mention linking, explicit closing keywords, idempotent.
- Inbox: `ticket.escalated` (deskcraft) and `contract.deadline` (contractcraft) become issues; `capacity.changed` (teamcraft) is shown in status. All three verified end to end.
- 17 MCP tools (TOOLS.md, generated and test-checked), CLI with board view.

Boundaries

- No hosted sync, no GitHub/GitLab issue sync in v1 (gitops-mcp covers GitHub).
- Git is read-only; taskcraft never writes to a repo.
- No time tracking (teamcraft) and no customer communication (deskcraft).

Gaps (honest)

- No egui window; the board is CLI text and JSON.
- No comments or attachments on issues.
- No sprints / cycles; capacity is shown, not planned against.
- Git sync on demand only (no hook or watcher).
- Synthetic data only.

Next

1. egui app: board lanes, issue detail with history, milestone burn-up.
2. Comments on issues.
3. Use teamcraft capacity in milestone planning.

Complete MCP Tool & CLI Verb Reference

Generated by `taskcraft tools --markdown`. Do not edit by hand: a test fails when this file is out of date. Every tool is also a CLI verb (`taskcraft --help`). Dotted names (`app.verb`) are accepted.

Tool	What it does	Required	Optional
`task_status`	Tracker status: counts (open, in progress, urgent, overdue), projects, demo flag.	-	-
`task_project_create`	Create a project with a 2-10 char key (issues become KEY-n). Optional repo_path enables git sync.	`key`, `name`	`repo_path`
`task_project_list`	List projects.	-	-
`task_issue_create`	Create an issue.	`project`, `title`	`assignee`, `description`, `due_date`, `estimate`, `labels`, `milestone`, `priority`, `status`
`task_issue_list`	List issues, urgent first. Finished issues are hidden unless include_finished or an explicit status filter.	-	`include_finished`, `label`, `limit`, `milestone`, `priority`, `project`, `status`, `text`
`task_issue_show`	One issue with relations, open blockers, linked commits and full history.	`issue`	-
`task_issue_update`	Change fields. Pass null to clear estimate/assignee/milestone/due_date. labels replaces the set.	`issue`	`assignee`, `description`, `due_date`, `estimate`, `labels`, `milestone`, `priority`, `title`
`task_issue_move`	Change status. Moving to in_progress/in_review/done is refused while an open issue blocks it, unless force=true.	`issue`, `status`	`force`
`task_relation_add`	Relate two issues. 'blocks' edges may not form a cycle.	`source`, `kind`, `target`	-
`task_relation_remove`	Remove a relation.	`source`, `kind`, `target`	-
`task_milestone_create`	Create a milestone in a project.	`project`, `title`	`due_date`

Tool	What it does	Required	Optional
`task_milestone_list`	Milestones with progress (issues and points done/total).	-	`project`
`task_milestone_close`	Close a milestone.	`project`, `title`	-
`task_board`	Kanban lanes for a project: cards per status, points, blocked flags.	`project`	-
`task_git_sync`	Read the project repo's git log, link commits mentioning KEY-n, close issues on 'closes/fixes/resolves KEY-n'. Needs --allow-path covering the repo.	`project`	`max`
`task_inbox_drain`	Turn events from other craft apps (deskcraft ticket.escalated, contractcraft contract.deadline) into issues; teamcraft capacity.changed is stored per week and shown in task_status.	-	-
`task_demo_seed`	Fill an EMPTY tracker with a labelled demo project.	-	-